Applied Programming · Assignment 2

Build and Verify a Path-Geometry Package

Functions, tests, NumPy arrays, and reproducible evidence

Engineering programming problem. Complete a reusable numerical package that measures and interpolates two-dimensional geometric paths. The package must behave consistently for tuples, lists, and NumPy arrays, reject invalid public input, preserve caller-owned arrays, and produce evidence that another person can rerun from one identified Git revision.

The path is already given. You are not implementing a planner, obstacle model, collision checker, robot controller, or physics simulator in this assignment. The supplied visualization displays the arrays returned by your functions; it is supporting evidence rather than the main implementation task.

Provided and protected	You complete	Final evidence
package structure, function names and signatures	four function bodies in `path_geometry.py`	published and student tests
published tests and environment verifier	at least six independent tests	JSON summary and SVG preview
review example, demo, visualization and artifact scripts	engineering note	wheel, private repository and commit ID

The five public objects are `PathInputError`, `segment_length`, `interpolate_segment`, `path_length`, and `path_lengths`. Their names, signatures, units, shapes, exception type, and no-mutation behavior are part of the fixed contract.

Files you may change. `src/ap_week02_path/path_geometry.py`, `tests/test_student_evidence.py`, and `artifacts/engineering_note.md`. Do not change protected tests, demo, visualization, environment scripts, package metadata, or public names to make a failure disappear.

Step 1 - Create the private repository and verify access

Open the public course template below. Use Use this template to create `applied-programming-w02-<student-id>` under your own account. Set visibility to Private, invite the instructor account `SSUMechE`, and confirm that the invitation has been accepted. Clone this private repository; do not use the public template as your submission origin.

```
https://github.com/SSUMechE/applied-programming-2026-week02-starter
```

Step 2 - Install and verify before editing

```
conda activate applied-programming-2026
python -m pip install -r requirements.txt
python -m pip install -e . --no-build-isolation
python scripts/verify_environment.py
git remote -v
```

The verifier must end with `[PASS] Week 2 path-geometry environment is ready.` Both remote lines must point to the private repository you created. Resolve environment or access problems before changing source code.

Step 3 - Review familiar Python data flow

```
python -m ap_week02_path.review_data_flow
```

Trace one point as a tuple, one path as a list of tuples, unpacking, slicing, `zip`, a loop, a list comprehension, and conversion to a NumPy array. This protected example reviews syntax but does not contain the function solutions.

Step 4 - Record the intentional baseline

```
python scripts/run_baseline.py
```

The starter contains four deliberate `NotImplementedError` bodies. The verified baseline is `42 failed, 6 passed`. Save that final line and the first failing test name. Do not edit `tests/test_published_contract.py`.

Step 5 - Complete the four functions in dependency order

Work on one behavior at a time and run the focused published suite after each change. Validation must accept finite integer or floating-point tuple, list, or array input and reject Boolean, complex, string, object, empty, wrong-shaped, and non-finite input with `PathInputError`.

Order	Function and exact required behavior
1	`segment_length(start_xy, goal_xy)`: validate two `(2,)` points and return one nonnegative Python `float`. Identical endpoints return `0.0`.
2	`interpolate_segment(start_xy, goal_xy, num_samples)`: require a Python or NumPy integer `num_samples >= 2` (Boolean is invalid); return `float64` shape `(M,2)` including both endpoints.
3	`path_length(path_xy)`: require finite shape `(N,2)` with `N >= 2`; sum consecutive Euclidean segment lengths and return a Python `float`.
4	`path_lengths(paths_xy)`: require finite shape `(B,N,2)` with `B >= 1`, `N >= 2`; use NumPy operations over path, waypoint, and coordinate axes and return `float64` shape `(B,)`. Do not use a Python `for`, `while`, or comprehension inside this function.

```
python -m pytest -q tests/test_published_contract.py -x
```

The public functions must not print, plot, write files, call the visualization, alter global state, or modify caller-owned arrays. Do not reshape an invalid array or clip a value to make it fit the contract.

> Implementation sequence. representation and shape validation → finiteness validation → numerical calculation → output type and shape. Keep the scalar meaning stable before writing the batch implementation.

Step 6 - Add independent tests

Add at least six top-level `test_...` functions to `tests/test_student_evidence.py`. They must use inputs different from the published examples and must collectively include all of the following evidence.

Required evidence	Minimum expectation
normal	one new segment, interpolation, or path result with an independently known relation
boundary	for example zero-length segment or the minimum permitted sample/path size
invalid	wrong shape, non-finite value, invalid `num_samples`, or a disallowed value type
endpoint	first and last interpolation rows equal start and goal
translation	one common coordinate offset preserves path length
batch or ownership	scalar-batch equivalence, row permutation, or no mutation

Use `np.isclose` or `np.allclose` with an explicit tolerance. Use `pytest.raises` for invalid input. A copy of a published case with different variable names does not count as independent evidence.

Step 7 - Generate numerical and visual evidence

```
python -m pytest -q
python -m ap_week02_path.demo
python -m ap_week02_path.visualize
python scripts/generate_artifacts.py
```

Confirm that the demo reports the scalar segment, interpolation shape and endpoints, one-path length, batch lengths, scalar-batch agreement, and unchanged inputs. Confirm that `artifacts/path_geometry_summary.json` and `artifacts/path_geometry_preview.svg` were regenerated from the current code.

Complete all five sections of `artifacts/engineering_note.md`: public validation layers; one normal and one boundary result with unit and tolerance; scalar-batch equivalence; translation-invariance and no-mutation evidence; and the boundary of what the supplied SVG does and does not establish.

Step 8 - Build and freeze the submitted revision

```
python -m pytest -q
python scripts/check_submission.py
python -m build --wheel --no-isolation
git status --short
git add src/ap_week02_path/path_geometry.py tests/test_student_evidence.py artifacts dist
git commit -m "Complete Week 2 path-geometry package"
git push
git status --short
git rev-parse HEAD
```

The final test run must pass all published tests. `check_submission.py` must also confirm at least six top-level `test_...` functions in the student test file and the required evidence files. The wheel must appear under `dist/`. After pushing, `git status --short` must print no entries. Record the complete 40-character commit identifier; a branch name, screenshot, or the phrase 'latest version' does not identify the evaluated source revision.

Step 9 - Submit through the LMS

LMS field	Required value
1	private GitHub repository URL
2	full 40-character commit identifier from `git rev-parse HEAD`
3	confirmation that `SSUMechE` has active access

Completion conditions. environment verifier PASS; published tests pass; at least six top-level student `test_...` functions pass; four function contracts complete; numerical and SVG artifacts regenerated; engineering note complete; wheel built; private origin correct; `SSUMechE` access active; pushed revision clean; LMS fields submitted.

Never submit a password, personal access token, two-factor code, or recovery code. The LMS timestamp and stated commit identify the grading boundary. Later commits are not part of the submission unless a new commit identifier is submitted under the announced policy.